



中山大学计算机学院

人工智能

本科生实验报告

(2022 学年春季学期)

课程名称: Artificial Intelligence

教学班级	20240574	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇

一、实验题目

2.4 利用遗传算法求解 TSP 问题

□ 利用遗传算法求解 TSP 问题

- 在National Traveling Salesman Problems (uwaterloo.ca) (<https://www.math.uwaterloo.ca/tsp/world/countries.html>) 中任选两个 TSP问题的数据集。
- 建议:
 - 对于规模较大的TSP问题,遗传算法可能需要运行几分钟甚至几个小时的时间才能得到一个比较好的结果.因此建议先用城市数较小的数据集测试算法正确与否,再用城市数较大的数据集来评估算法性能。
 - 由于遗传算法是基于随机搜索的算法,只运行一次算法的结果并不能反映算法的性能.为了更好地分析遗传算法的性能,应该以不同的初始随机种子或用不同的参数(例如种群数量,变异概率等)多次运行算法,这些需要在实验报告中呈现。

二、实验内容

1. 算法原理

遗传算法是一种模拟自然选择和遗传机制的优化算法。它是一种启发式的搜索算法,通过模拟生物进化的过程来解决优化问题。遗传算法的基本思想是通过迭代地选择、交叉和变异来生成新的解,直到找到一个较优解。

遗传算法的基本步骤包括:

1. 初始化种群:



随机生成一组候选解，这些解构成初始种群。

2.选择:

根据适应度函数选择个体，适应度较高的个体有更大的机会被选为父代。适应度是个体解的优劣标准，通常用问题的目标函数来衡量。

3.交叉:

从父代中选出两个个体，通过交换它们的基因（也就是路径部分）生成新的个体（子代）。交叉操作模拟生物遗传中的基因交换。

4.变异:

在子代中进行小范围的随机变化，模拟生物中的基因突变。变异有助于保持种群的多样性，避免陷入局部最优。

5.替换:

根据某些策略（如精英选择策略），将部分新生成的子代替代老一代的个体，更新种群。

6.终止条件:

达到设定的迭代次数或满足某些停止条件时，算法终止，输出最佳解。

在这道题中的应用:

遗传算法被应用于旅行商问题（TSP, Travelling Salesman Problem），即给定一组城市和它们之间的距离，要求找到一条最短的路径，使得从一个起点城市出发，经过所有城市一次并最终回到起点城市。

主要步骤在代码中的应用:

1.初始化种群:

首先生成一个种群，每个种群个体即路径，是城市的一个排列，通过打乱城市顺序来生成随机路径。

表示方法一：随机排列

■ 对于n个城市的TSP问题，解表示为1-n的排列

染色体表示

5	4	6	9	2	1	7	8	3
---	---	---	---	---	---	---	---	---

环游顺序: 5-4-6-9-2-1-7-8-3-5

2.选择操作:

根据路径的长度即适应度，来选择父代。路径越短，适应度越高，越有可能成为父代。

3.交叉操作:

从父代中选择两个个体，通过交换部分基因即城市的顺序生成新的子代。

部分映射交叉（PMX）的步骤:

随机选择两个交叉点：随机选择两个下标 s 和 t ，它们定义了交叉的区间（即子串）。

交换两个子串：将两个父代个体中的交叉区间交换，生成两个子代。

处理映射关系：为了避免重复，部分映射交叉需要在交换的过程中，映射父代的基因。

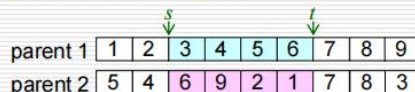
生成新的子代染色体：根据映射关系调整子代，保证没有重复元素。



1.4 高级搜索

3. 旅行商问题——部分映射交叉

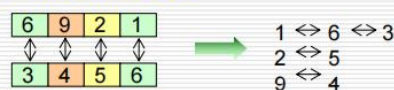
step 1: 随机选择下标 s, t



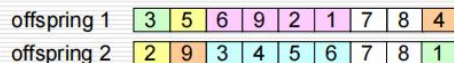
step 2: 交叉子串



step 3: 确定映射关系



step 4: 生成后代染色体



算法 染色体交叉操作

输入: 染色体 v_1, v_2 及其长度 l

- 1: $R \leftarrow \emptyset$
切割染色体
- 2: 随机生成两个下标 s, t , 满足 $0 < s < t < l - 1$
交叉染色体
- 3: $v'_1 \leftarrow [v_1[0], \dots, v_1[s-1], v_2[s], \dots, v_2[t], v_1[t+1], \dots, v_1[l-1]]$
- 4: $v'_2 \leftarrow [v_2[0], \dots, v_2[s-1], v_1[s], \dots, v_1[t], v_2[t+1], \dots, v_2[l-1]]$
基因映射
- 5: 依据 $[v_1[s], \dots, v_1[t]]$ 与 $[v_2[s], \dots, v_2[t]]$ 建立映射关系
生成后代染色体
- 6: 依据映射关系修改 v'_1, v'_2 非交叉部分

其他交叉操作: order crossover (OX), cycle crossover (CX), position-based crossover, order-based crossover等. 感兴趣的同学可自行尝试.

50

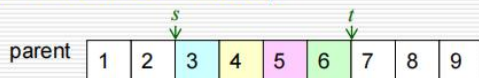
4. 变异操作:

随机交换子代路径中的两个城市位置, 以进行变异。变异操作是为了增加解的多样性, 避免算法陷入局部最优。

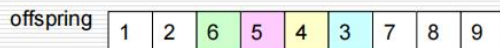
1.4 高级搜索

3. 旅行商问题——倒置变异

step 1: 随机选择下标 s, t



step 2: 将 $s-t$ 中间部分倒置



算法 染色体变异操作

输入: 染色体 v 及其长度 l

- 1: $R \leftarrow \emptyset$
切割染色体
- 2: 随机生成两个下标 s, t , 满足 $0 < s < t < l - 1$
倒置
- 3: $v' \leftarrow [v[0], \dots, v[s-1], v[t], v[t-1], \dots, v[s+1], v[s], v[t+1], \dots, v[l-1]]$
- 4: return v'

51

5. 迭代与优化:

通过多代的迭代，不断更新种群，最终得到最短的路径，即最优解。

6. 适应度路径长度计算:

计算路径的总长度，来表示种群的适应度。路径的总长度由城市之间的欧几里得距离计算得出。

2. 伪代码

导入 `numpy, pandas, matplotlib, math, random`

初始化图形参数以支持中文显示

函数 `read_data()`:

- 读取 CSV 文件数据

- 提取城市名称、x 坐标、y 坐标

- 绘制城市位置图并标注城市名称

- 返回城市名称、x 坐标、y 坐标

函数 `distances(城市名称, x 坐标, y 坐标)`:

- 初始化城市总数

- 初始化一个全 0 的距离矩阵

- 对每对城市计算欧几里得距离并填充矩阵

- 返回城市总数、距离矩阵

函数 `path_length(路径, 起始城市)`:

- 初始化距离为 0

- 计算从起点到第一个城市的距离

- 对路径中每两个城市之间的距离进行累加

- 返回路径的总长度

函数 `get_result(种群, 起始城市)`:

- 计算每个路径的总长度并排序

- 返回最短路径和长度

函数 `selection(种群, 保留率, 生命强度, 起始城市)`:

- 计算每个路径的总长度并排序

- 保留适应性强的种群

- 根据生命强度随机选择适应性较差的个体

- 返回选择的父代种群

函数 `pmx_crossover(父代种群, 种群个体数)`:

- 随机选择父代个体进行交叉

- 选择两个交叉点并交换基因

- 使用 PMX 映射方法避免重复基因

- 返回子代种群

函数 `mutation`(子代种群, 变异率):

根据变异率随机交换路径中的两个城市位置
返回变异后的子代种群

函数 `mutation_reverse`(子代种群, 变异率):

根据变异率随机倒置路径中的部分城市
返回变异后的子代种群

函数 `plt_magin`(迭代次数, 路径长度, 最优路径, 起始城市, 城市名称, x 坐标, y 坐标):

打印当前迭代的最优路径长度
绘制最优路径的城市位置图
绘制每代最优路径的变化图

函数 `GA_TSP`(源点, 种群个体数, 改良次数, 进化次数, 保留率, 生命强度, 变异率):

读取城市数据并计算城市距离矩阵
初始化路径列表并随机生成初始种群
对每代种群进行选择、交叉、变异等操作
记录每代最优路径的长度并可视化
输出最终结果

3. 关键代码展示（带注释）

1. 读取城市的.tsp 格式文件数据

```
def read_map():  
    city_name = []  
    city_x = []  
    city_y = []  
    with open('ja9847.tsp', 'r') as file:  
        lines = file.readlines()  
        reading_coords = False  
        for line in lines:  
            #跳过注释行  
            if line.startswith('COMMENT') or not line.strip():  
                continue  
            #开始读取节点坐标部分  
            if line.strip() == 'NODE_COORD_SECTION':  
                reading_coords = True  
                continue  
            #结束读取节点坐标部分  
            if line.strip() == 'EOF':  
                break  
            #读取坐标数据  
            if reading_coords:
```



```
parts = line.split()

if len(parts) == 3:

    city_name.append(parts[0])

    city_x.append(float(parts[1]))

    city_y.append(float(parts[2]))

return city_name, city_x, city_y
```

2. 计算城市之间的距离

计算不同城市间的距离矩阵

```
def cal_distances(city_name, city_position_x, city_position_y):

    global city_distance # 声明城市之间距离为全局变量

    city_num = len(city_name) # 城市数量

    city_distance = np.zeros([city_num, city_num]) # 初始化城市距离矩阵 np.zeros 创建一个全 0 的矩阵，参数为矩阵的行和列

    for i in range(city_num): # 计算两城市之间的欧几里得距离

        for j in range(city_num):

            city_distance[i][j] = math.sqrt((city_position_x[i] - city_position_x[j]) ** 2 + (city_position_y[i] - city_position_y[j]) ** 2)

    return city_num, city_distance
```

3. 计算路径的总长度花费

计算一条路径的总长度

```
def cal_length(path, origin): # 参数：具体路径，出发点

    distance = 0

    distance += city_distance[origin][path[0]] # 从起点到第一个城市的距离

    for i in range(len(path)):

        if i == len(path) - 1:

            distance += city_distance[origin][path[i]] # 从最后一个城市回到起点的距离

        else:

            distance += city_distance[path[i]][path[i + 1]] # 计算城市间的路径距离

    return distance
```

4. 找种群的最优个体

代种群最优个体

```
def get_result(population, origin):

    sorted_paths = [[cal_length(path, origin), path] for path in population] # 存储每个路径的长度和路径

    sorted_paths = sorted(sorted_paths) # 按路径长度升序排列取出最优路径

    return sorted_paths[0][0], sorted_paths[0][1] # 路径，长度
```

5. 找到父代种群

选择父代种群

```
def selection(population, retain_rate, live_rate, origin): # 参数：种群，适者比例，生命强度，出发点

    sorted_paths = [[cal_length(path, origin), path] for path in population]

    sorted_paths = [path[1] for path in sorted(sorted_paths)]

    retain_num = int(len(sorted_paths) * retain_rate) # 对应适者比例的适应性强的个体数量

    parents = sorted_paths[: retain_num] # 保留适应性强的前 retain_num 个体 成为父代的一部分以便繁殖
```




```
for i in sorted_paths[retain_num:]: #对于适应性弱的染色体

    if random.random() < live_rate: #随机数小于生命强度，则保留该个体成为父代的一部分以便繁殖

        parents.append(i)

return parents
```

6. 部分映射交叉

#部分映射交叉

```
def pmx_crossover(parents, population_num):

    children_num = population_num - len(parents) #计算要生成子代的个数

    children = []

    while len(children) < children_num:

        #在父母种群中随机选择父母

        male_index = random.randint(0, len(parents) - 1)

        female_index = random.randint(0, len(parents) - 1)

        if male_index != female_index:

            male = parents[male_index]

            female = parents[female_index]

            #随机选择两个下标 s 和 t

            s = random.randint(0, len(male) - 1)

            t = random.randint(s, len(male) - 1)

            if s>t:#确保 s 和 t 不相等，且 s<t

                s, t = t, s

            #交换两个父代个体中的子串

            child1 = male[:s] + female[s:t+1] + male[t+1:]

            child2 = female[:s] + male[s:t+1] + female[t+1:]

            #通过映射关系调整交叉后的路径，避免重复

            child1 = pmx_map(child1, male[s:t+1], female[s:t+1], s, t)

            child2 = pmx_map(child2, female[s:t+1], male[s:t+1], s, t)

            #将生成的子代添加到子代列表中

            children.append(child1)

            children.append(child2)

    return children
```

#映射来调整子代的顺序，且确保没有重复元素

```
def pmx_map(child, segment1, segment2, s, t):

    i=0

    while i<len(child):

        if s<=i<=t:#跳过索引范围 [s, t] 这一部分不需要映射

            i+=1

        else:

            if child[i] in segment2: #找到映射关系，并替换

                index = segment2.index(child[i])

                child[i] = segment1[index]

            #不需要+1 继续检查替换后的元素是否在 segment2 中

            i+=1
```



```
    else:
        i+=1 #处理下一个值
    return child
```

7.倒置变异

```
#倒置变异
def mutation_reverse(children, mutation_rate): #参数: 孩子种群, 变异率
    for i in range(len(children)):
        if random.random() < mutation_rate:
            child = children[i]
            #随机选择下标 s 和 t
            s = random.randint(0, len(child) - 2)
            t = random.randint(s + 1, len(child) - 1)
            #将 s 到 t 中间部分倒置
            child[s:t+1] = child[s:t+1][::-1]
            #保存变异后的子代
            children[i] = child
    return children
```

8.遗传算法

```
#遗传算法总流程
def GA_TSP(origin, population_num, improve_count, iter_count, retain_rate, live_rate, mutation_rate):
    #源点, 种群个体数, 改良迭代数, 进化次数, 适者概率, 生命强度, 变异率

    city_name, city_position_x, city_position_y = read_map()
    city_num= cal_distances(city_name, city_position_x, city_position_y)
    list = [i for i in range(city_num)]
    list.remove(origin) #去掉源点,只保留其他城市,假设从源点出发

    population = [] #种群的列表,存储每个个体的路径
    for i in range(population_num): #随机生成种群
        path = list.copy()
        random.shuffle(path)
        #path = improve(path, improve_count, origin) #优化 提高初始化种群多样性
        population.append(path)

    every_gen_best = [] #存储每一代最好的路径长度
    for i in range(iter_count):
        #选择繁殖个体群
        parents = selection(population, retain_rate, live_rate, origin)
        #交叉繁殖
        children = pmx_crossover(parents, population_num)
        #倒置变异
        children = mutation_reverse(children, mutation_rate)
        #更新种群
        population = parents + children
        distance, result_path = get_result(population, origin)
```



```
every_gen_best.append(distance)

#输出结果

plt_magin(i, distance, result_path, origin, city_name, city_position_x, city_position_y) #最终结果

plot_best_path_length(every_gen_best) #绘制每一代的最优路径长度变化曲线

plt.show()
```

三、 实验结果及分析

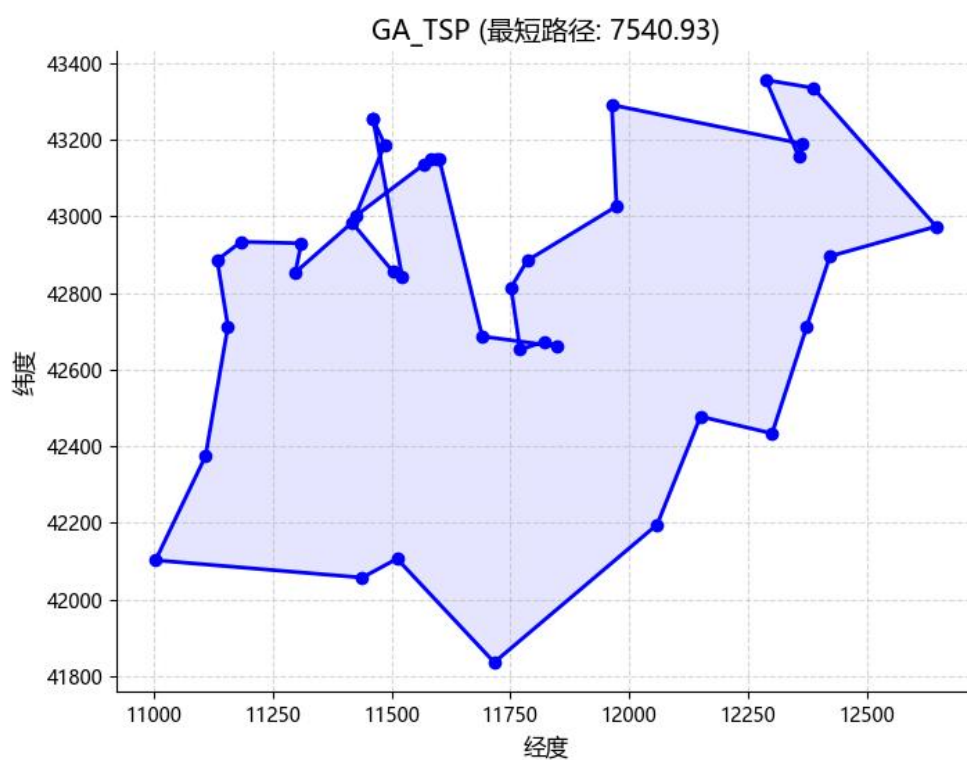
1. 实验结果展示示例（可图可表可文字，尽量可视化）

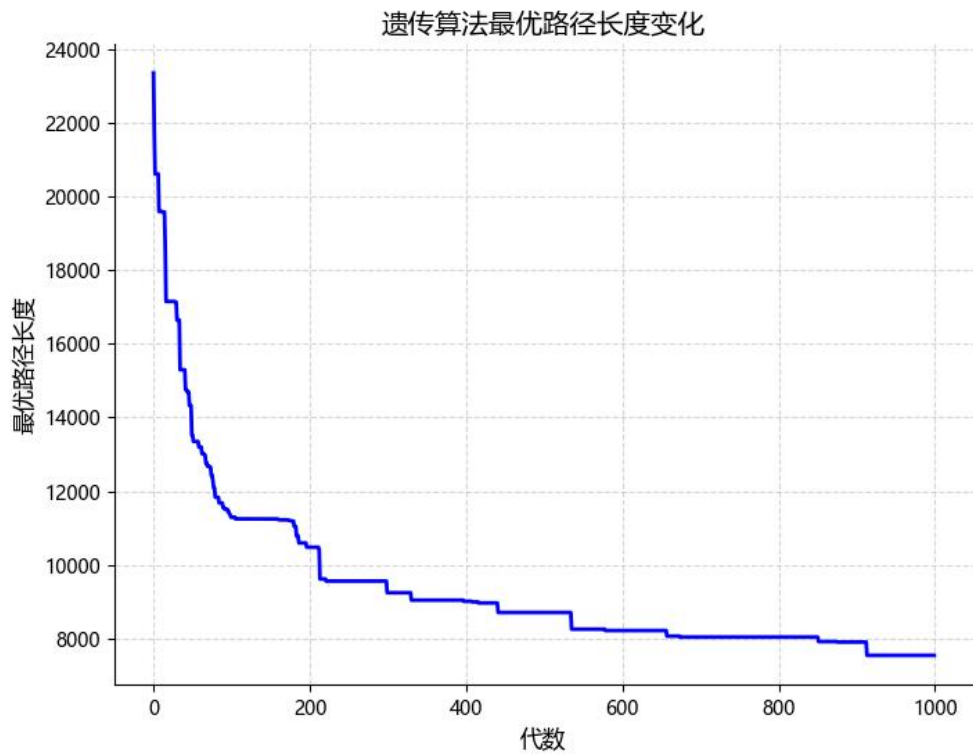
案例一：

38 locations in Djibouti

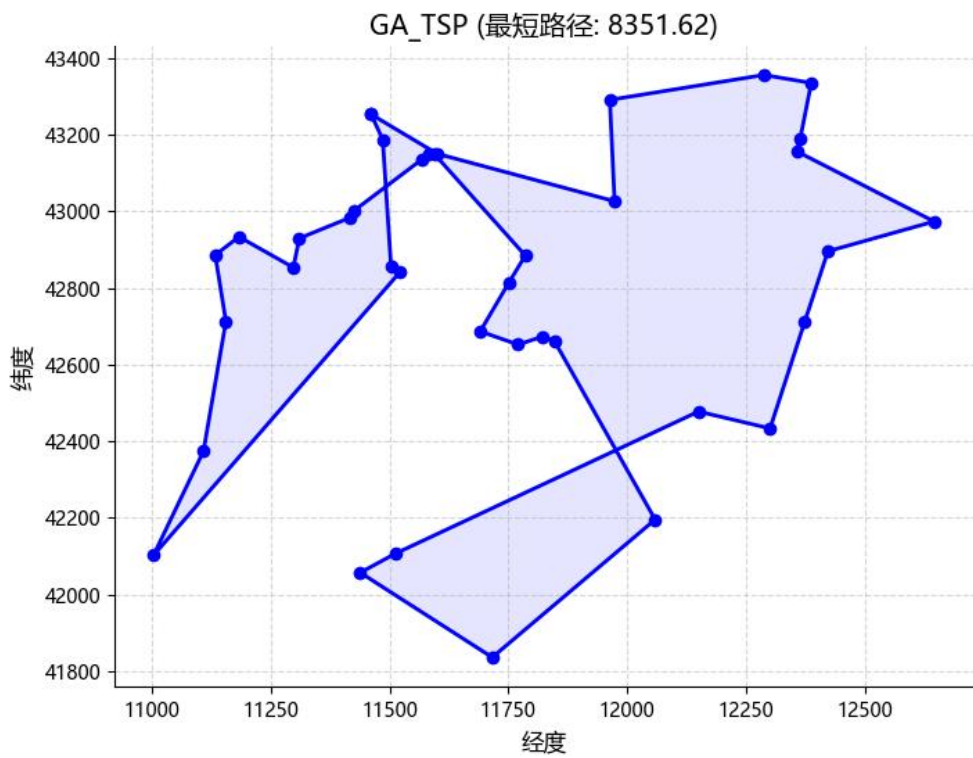
种群个数 300 进化次数 1000 适者概率 0.3 生命强度 0.5 变异率 0.01

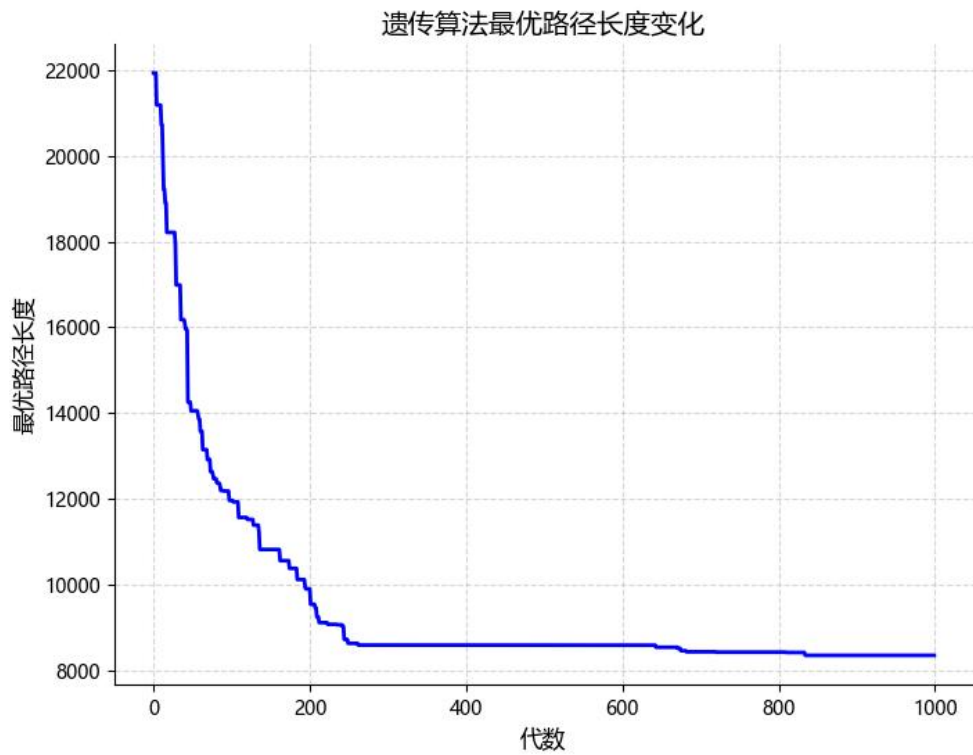
第一次：



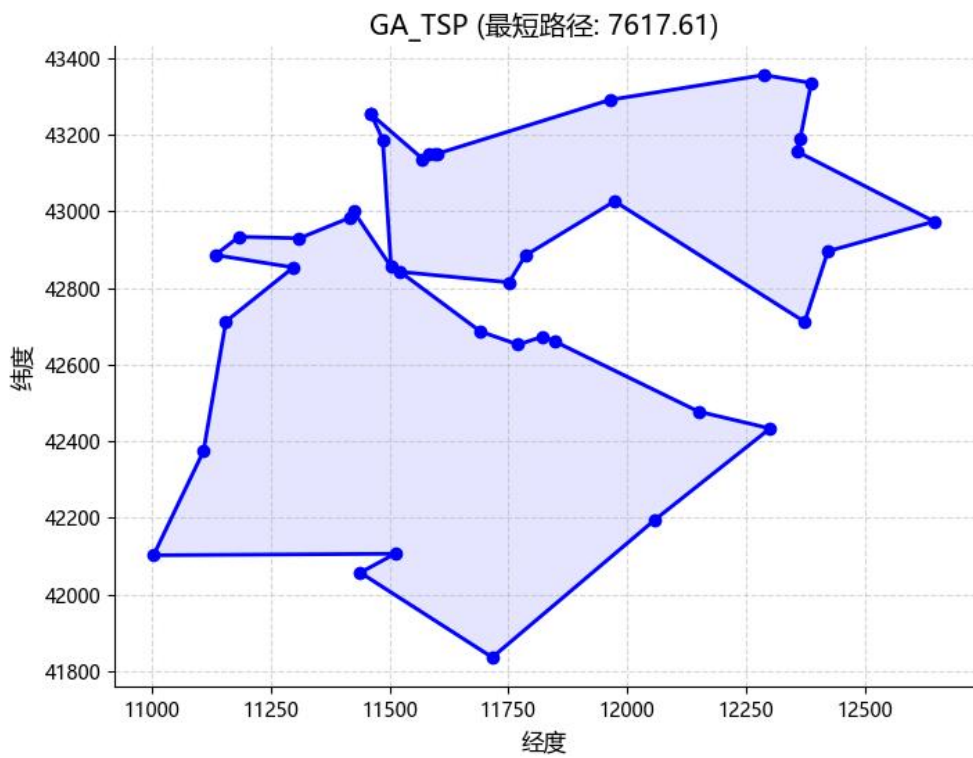


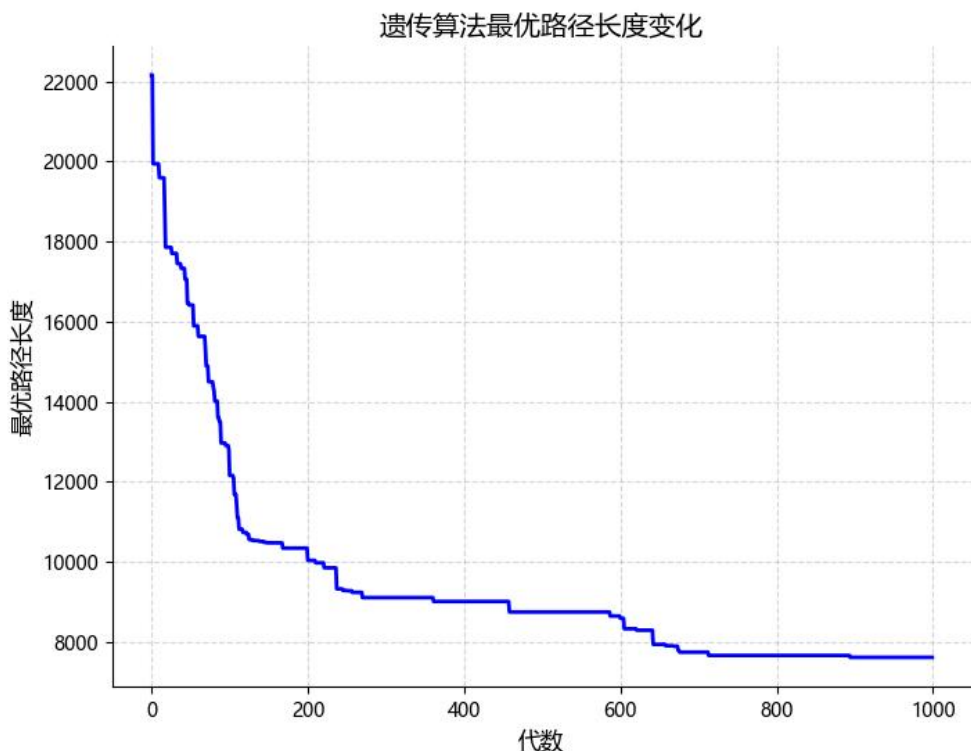
第二次





第三次





2. 评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

评测指标

最短路径长度：通过遗传算法得到的三个最短路径分别为 7540.93、8351.62、7617.61。这个指标反映了模型在不同进化过程中所获得的路径质量，可以看出每次进化的最短路径都在一定范围内变化，最短路径的结果没有出现较大的波动，这说明遗传算法能够在一定范围内逼近最优解。

收敛性：从最短路径变化不同，但大多数都在 200 次左右趋于答案，后面逐渐平缓，算法的收敛性较好。随着迭代次数的增加，路径长度的变化逐渐减小，说明遗传算法已经接近全局最优解。该特性表明，随着进化代数的增加，种群内部的个体趋于稳定，适应度趋向最大值，从而使得路径长度不再出现大幅度的波动。

适应度：在每一代中，通过适应度来衡量每个个体路径的优劣。适应度的变化通常表现为进化过程中逐渐提高的趋势，直到达到平稳状态。

遗传操作：变异率设置为 0.01，意味着在每一代中，大部分个体保持不变，只有少数个体会进行变异。这个设置能够防止过早收敛，保持种群的多样性，帮助算法更好地跳出局部最优解。交叉和变异共同作用，使得算法具有较好的探索和开发能力。

进化次数与最优解稳定性：进化次数设置为 1000 代，可以看到最短路径在 200 代左右趋于平稳，进一步证明在这次实验中，遗传算法在 200 代左右已经找到了一个近似的最优解，之后的进化主要是在局部范围内微调，优化结果逐渐收敛到平稳值。

分析

算法收敛性与稳定性：遗传算法的收敛性较好，经过约 200 代的演化，最短路径变化



趋于平稳，说明算法找到了一个稳定的解，并且避免了过度波动。这表明，算法在整体结构上较为稳定，不会受到单次变异或交叉的强烈影响。

适者生存与全局搜索：适者概率设定为 0.3 和变异率 0.01 能够保证良好的选择压力，同时不会使算法过早陷入局部最优解。变异操作的适当比例帮助搜索空间的多样性和广度，而适者生存则强化了强解的传递。

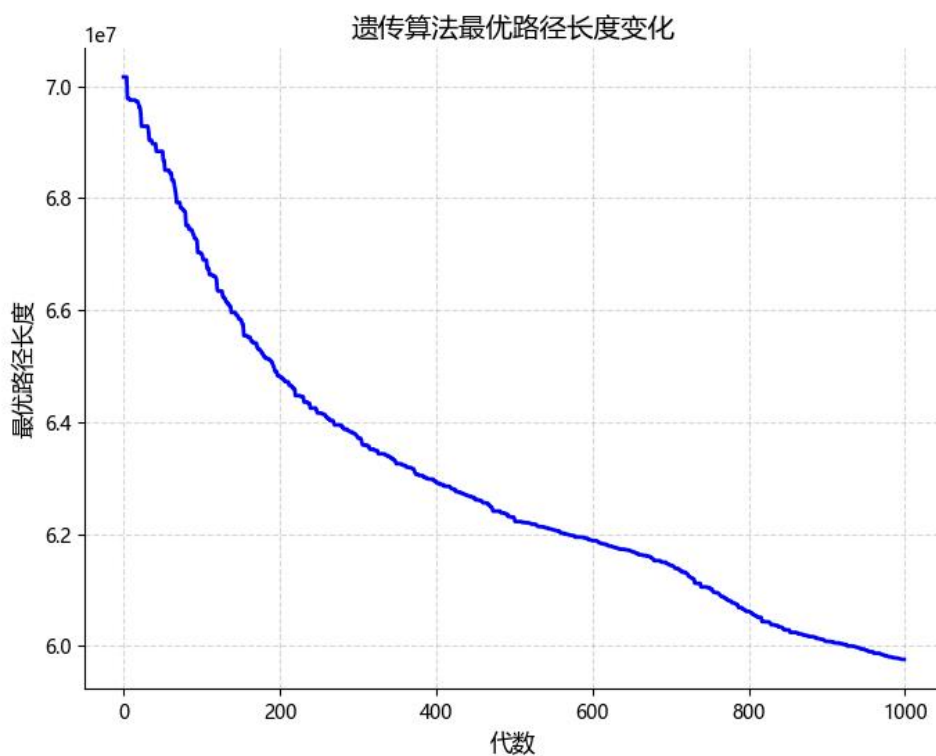
多次实验结果的差异性：三次实验结果之间（7540.93、8351.62、7617.61）差异较大，但仍处于合理范围内，说明遗传算法在求解过程中具有一定的随机性，最终的解会依赖于种群初始化和遗传操作的细微差异。

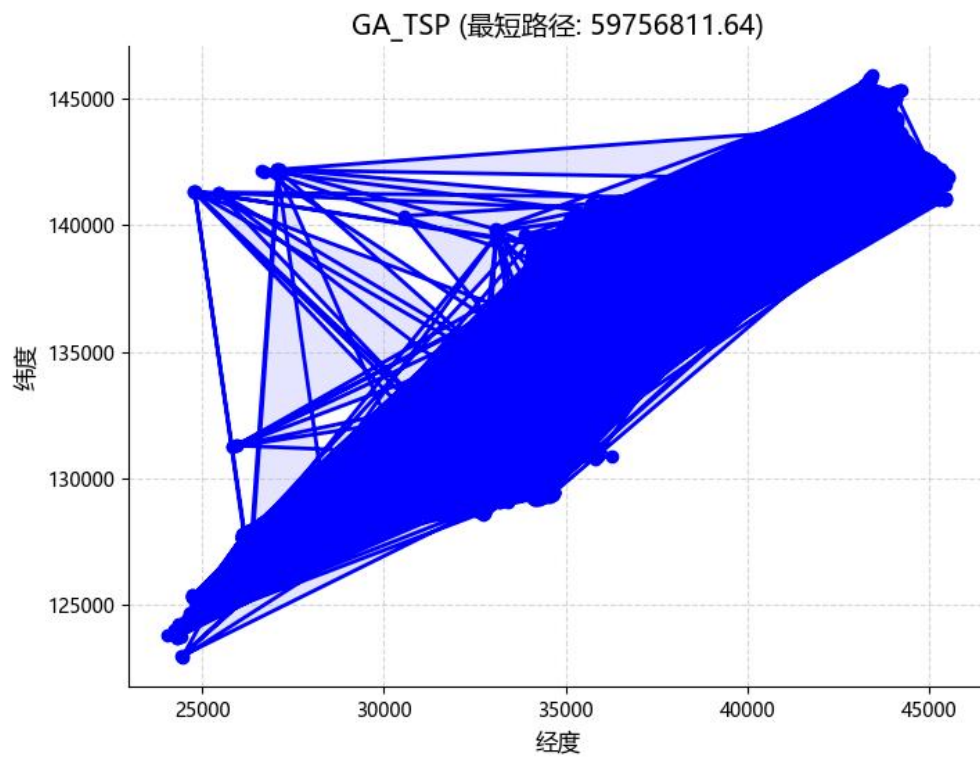
案例二：

9847 locations in Japan

同案例一参数

运行了三个小时，当城市数据量庞大时，运行时间久，但算法仍然保持可行性。

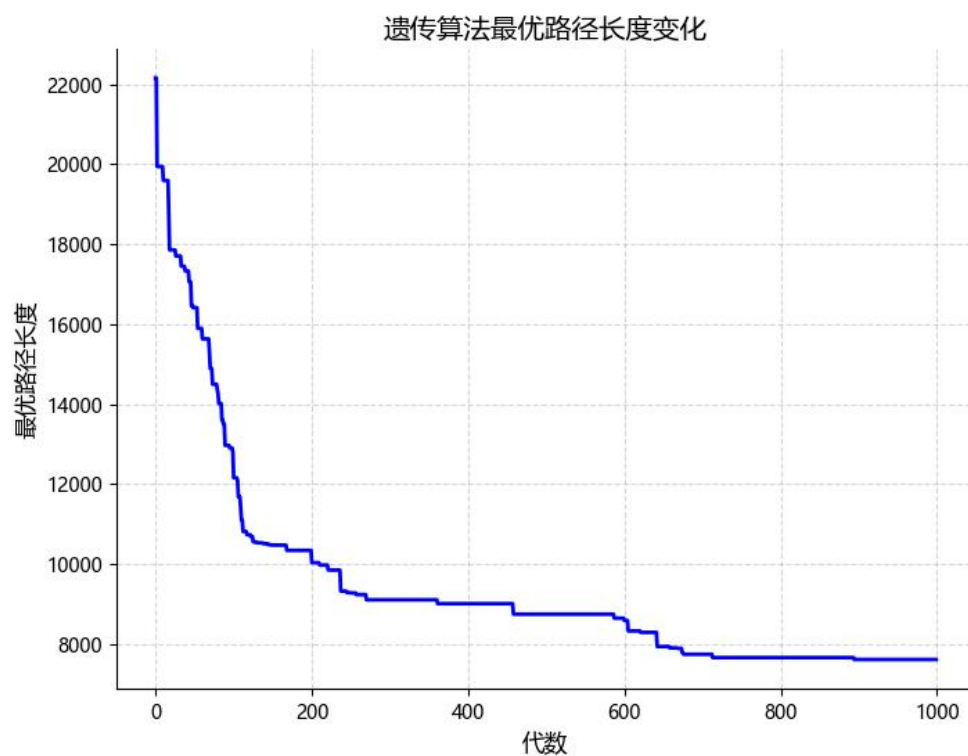
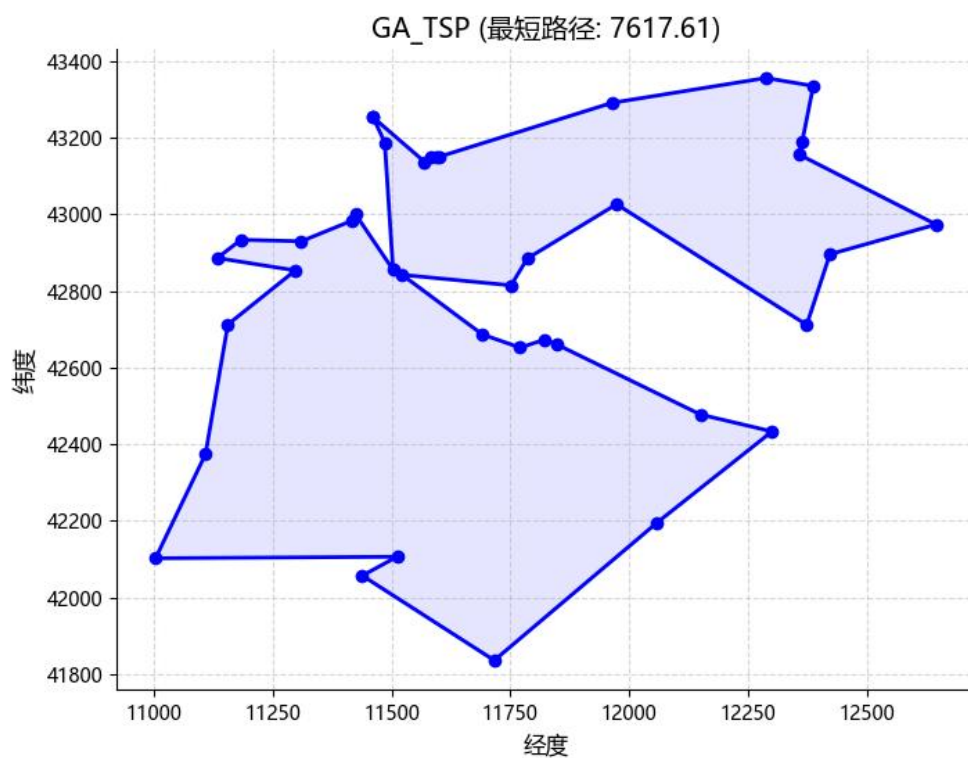




|-----如有优化，请重复 1，2，分析优化后的算法结果-----|

优化一：调整迭代次数

修改上述，迭代 10000 次，增加进化次数



增加进化次数到 10000 之后，得到了最短路径为 7617.61，这与之前在 1000 次进化下的最短路径（7617.61）相同。

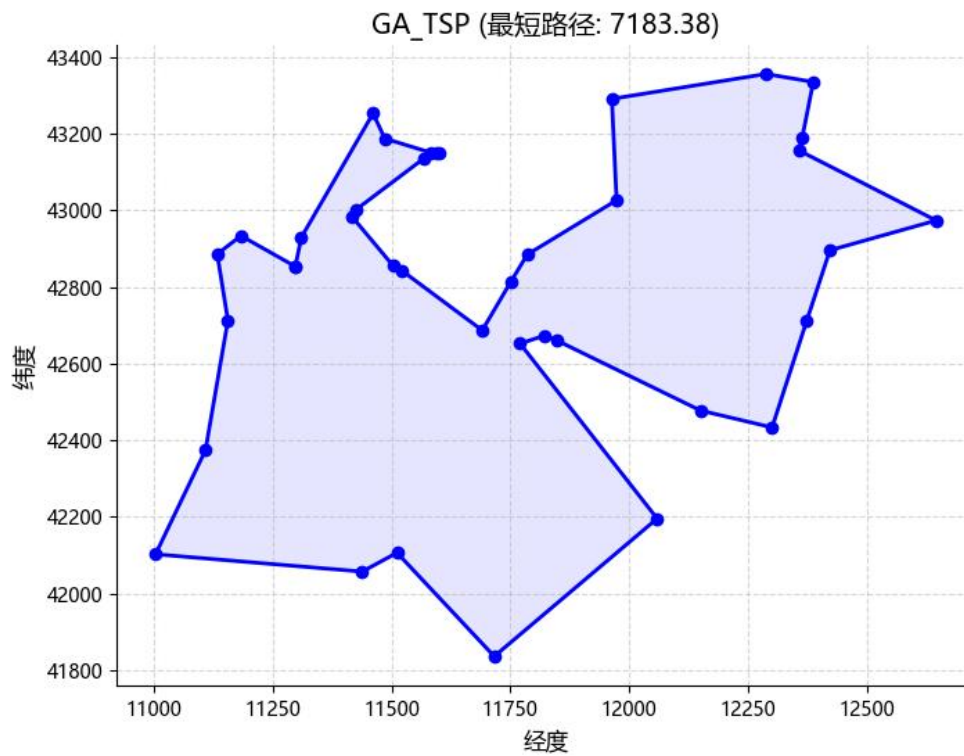
收敛性与早期最优解： 10000 次进化与 1000 次进化得到的最短路径一致，

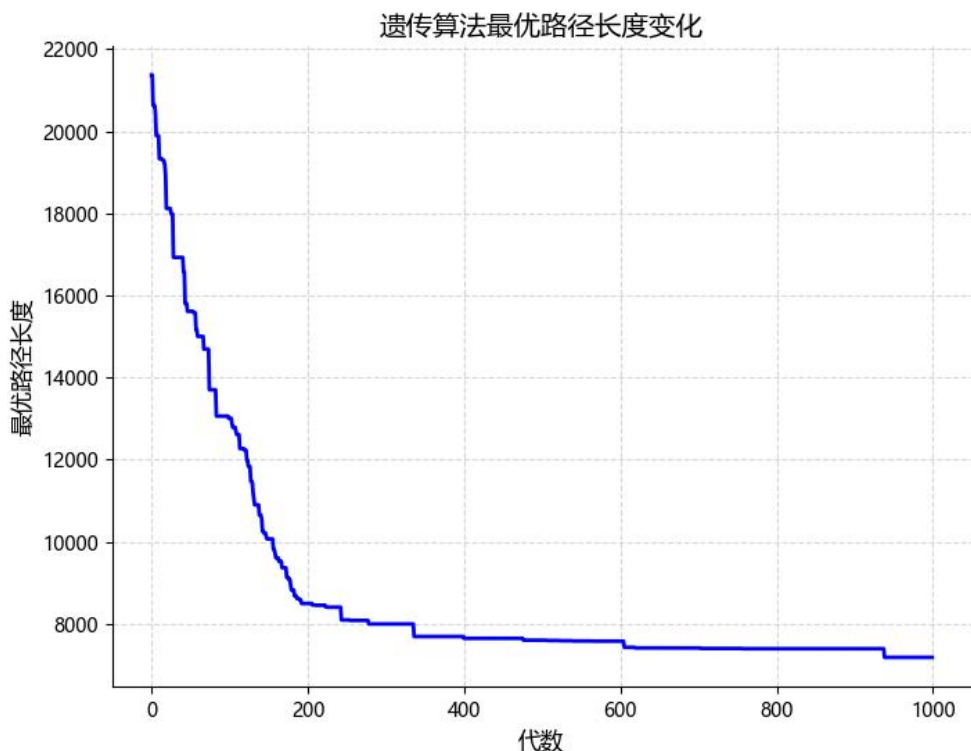


说明遗传算法在 1000 代左右就已经收敛到了一个相对较好的解。算法似乎已经在较短时间内找到了路径的局部最优，而增加更多的进化次数到 10000 次并未能有效突破这一局限。算法的收敛性非常强，且效率较高。

|-----如有优化，请重复 1，2，分析优化后的算法结果-----|

优化二：调整种群规模到 1000





评测指标展示和分析

最短路径长度：调整种群规模后，最短路径由 7617.61 降低到 7183.38，显示出种群规模增大后，算法能够找到更优的解。这表明增加种群规模提高了搜索空间的多样性，有助于避免局部最优解，从而找到更优的路径。

收敛性：从结果来看，增加种群规模使得遗传算法在更广泛的解空间中进行搜索种群规模较大有助于避免过早收敛到局部最优，从而提高了解的质量。

适应度变化：随着种群规模的增大，适应度的分布变得更加广泛，种群中出现了更多的优秀个体，进而推动了算法在搜索空间中更充分地探索。更大的种群规模能够增加选择和交叉操作中的多样性，避免陷入局部最优解。

算法效率：增加种群规模使得每代的计算量增大，进化过程变得更加复杂。然而，这种复杂性带来了更好的解，最终在更大的搜索空间中找到了更低的路径值（7183.38）。因此，虽然增加了计算资源和时间消耗，但从结果来看，获得了更好的解，这表明种群规模的增大在此情况下是有效的。

|-----如有优化，请重复 1，2，分析优化后的算法结果-----|

优化三：初始种群的优化，提高初始种群的质量

参数同案例一

```
#改良初代种群
def improve(path, improve_count, origin): #参数：具体路径，改良迭代次数，出发源点
    distance = cal_length(path, origin)

    for i in range(improve_count): #随机选择两个城市
        u = random.randint(0, len(path) - 1)
        v = random.randint(0, len(path) - 1)
```



```
if u != v: #交换两个城市 u v 的位置，生成新路径

    new_path = path.copy()

    t = new_path[u]

    new_path[u] = new_path[v]

    new_path[v] = t

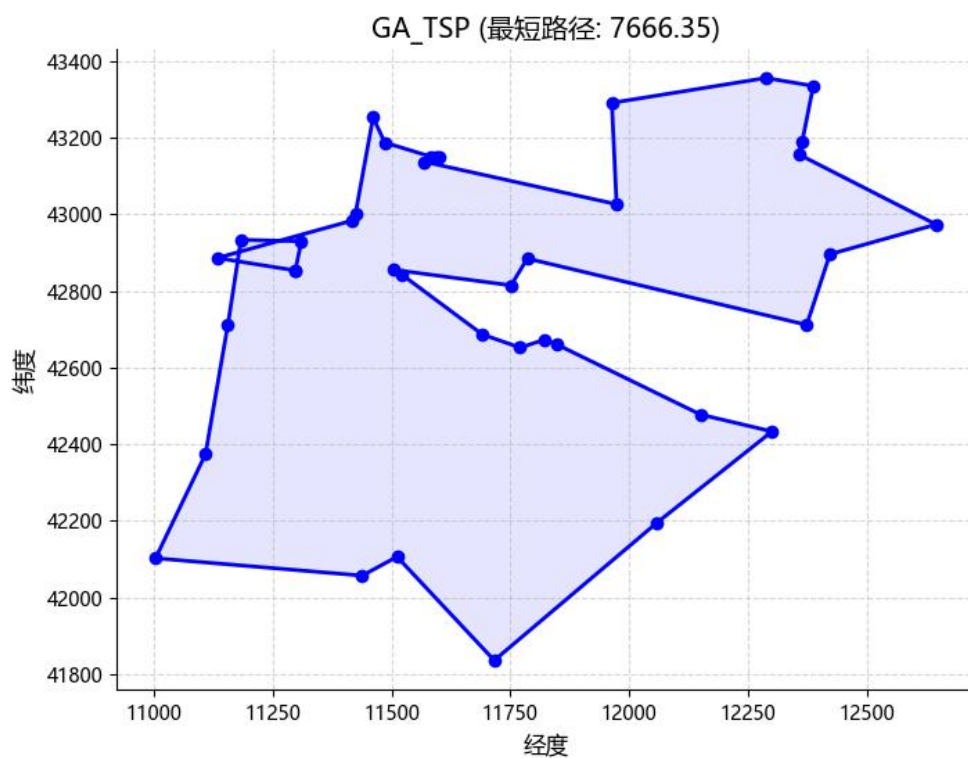
    new_distance = cal_length(new_path, origin)

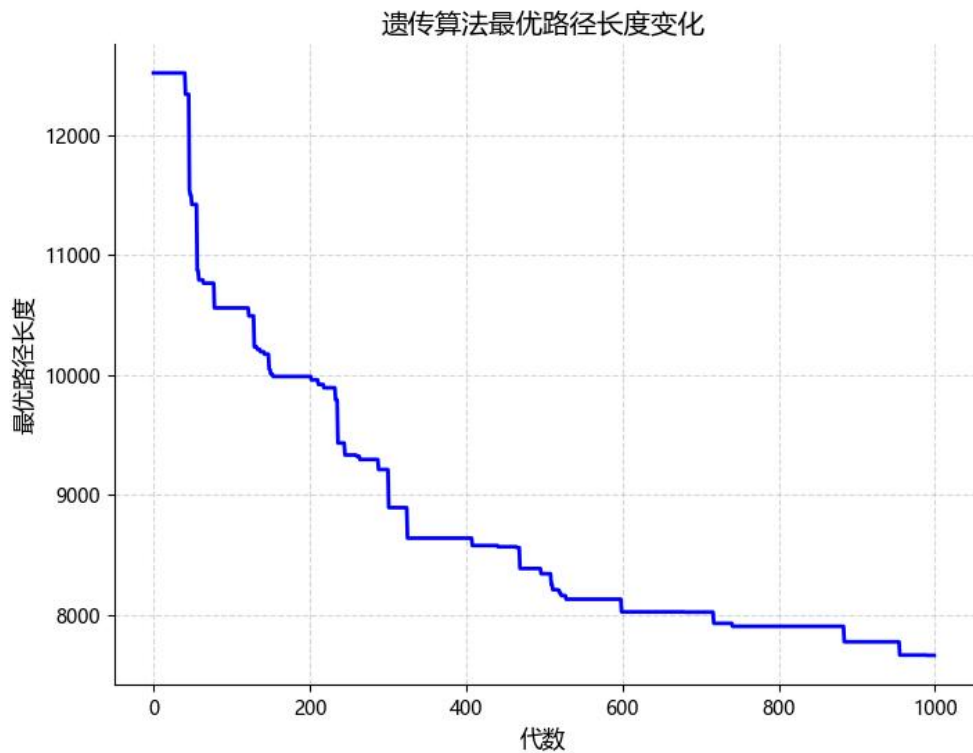
    if new_distance < distance: # 如果新路径更优，则保留该路径，提高种群质量

        distance = new_distance

        path = new_path.copy()

return path
```

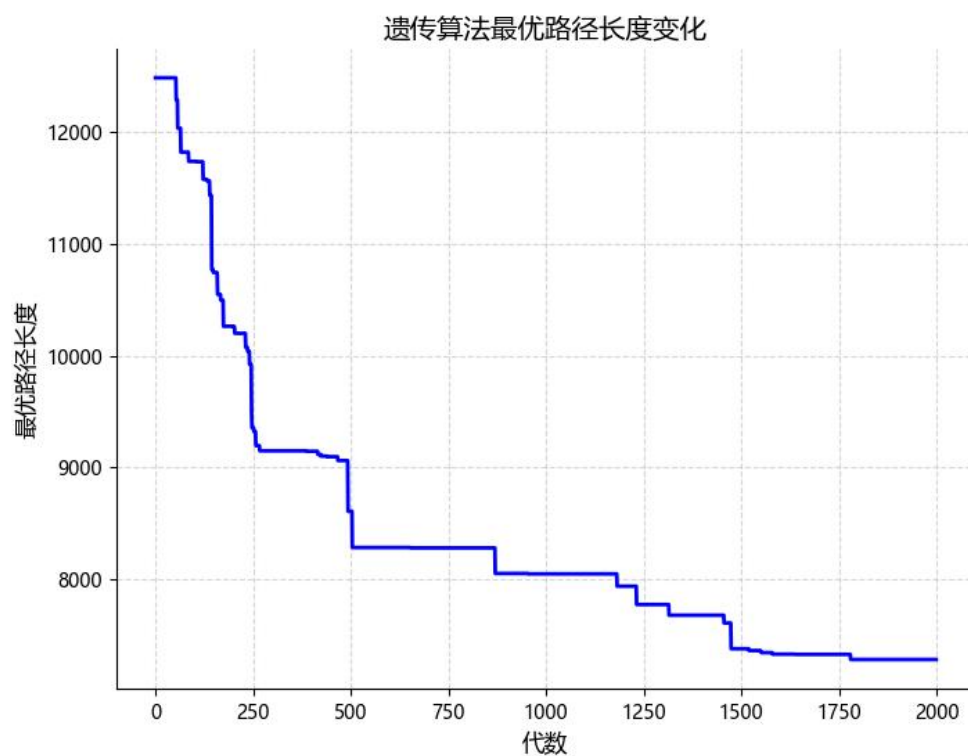
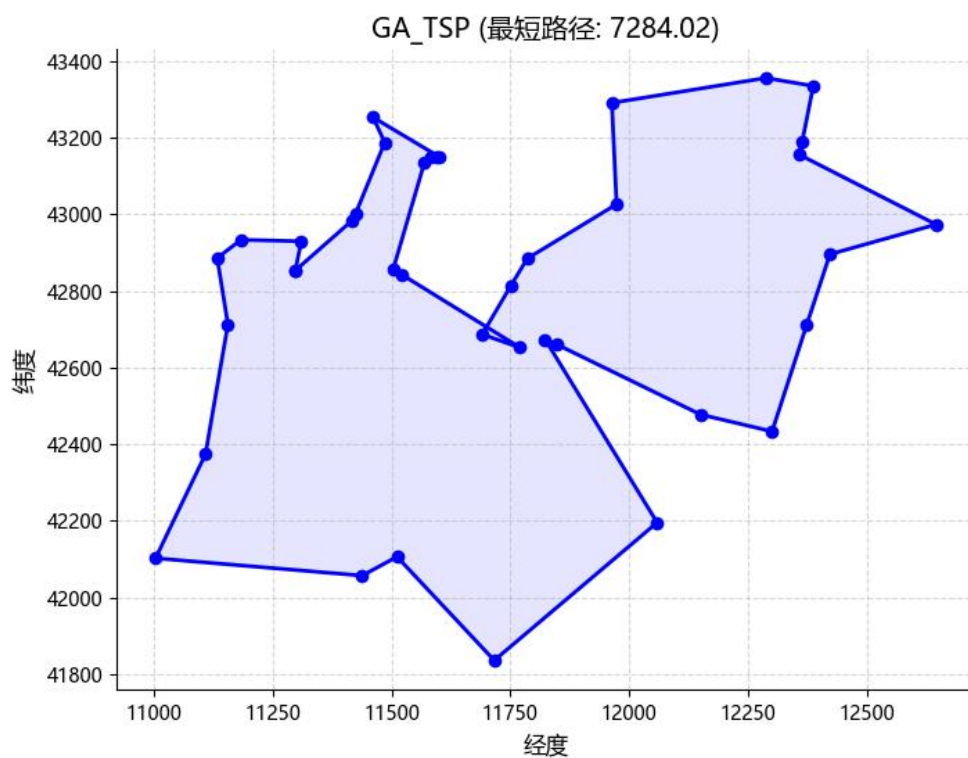




评测指标展示分析：

初始种群质量提升，但最短路径在很长时间内（直到约 900 代）还未趋于稳定。相比未使用 `improve()` 的情况，这次收敛更慢，但路径质量可能更好或更稳。

进一步优化：提高迭代次数从 1000 次到 2000 次，让算法找到最优解



结果表明，使用提升初代种群质量的算法收敛更慢，但路径质量在保证足够迭代次数的情况下会更好。

初始适应度水平较高，但由于个体相似性较强，遗传操作的能力下降，导致算法



在较长时间内处于“停滞”状态，直至变异或其他偶发事件打破局部最优，才引导路径向最优逼近。

对比前后优化的情况，需要考虑路径质量与探索能力的权衡，初代质量提高虽然理论上应更快收敛，但如果太过“优秀”且同质化严重，反而可能削弱算法的全局搜索能力，降低早期探索的多样性。

四、 思考题

1. 算法整体结构与求解思路清晰，层次分明

在本次 TSP 求解中，从地图数据解析、距离矩阵计算到路径初始化、改良、进化与变异操作，整体流程设计合理，代码结构清晰。遗传操作中的 PMX 交叉与倒置变异则确保了解的多样性与搜索空间的广泛探索。路径进化过程通过图像可视化，有效呈现了每代最优解的演化趋势，为调参和优化提供了明确依据。

2. 参数设置对收敛速度与解的质量具有显著影响

在调参过程中发现，种群规模、进化次数以及初始改良强度对最终路径质量有着重要影响。将种群个数由 300 提升到 1000 后，最短路径显著下降至 7183.38，说明更大的种群为算法提供了更丰富的基因多样性，增强了全局搜索能力。同时，将进化次数从 1000 增加至 10000，最短路径稳定在 7617.61，也体现出充足的迭代次数能让算法更充分地优化路径。然而，增加 `improve()` 的次数虽提升了初始种群质量，却导致最优路径迟至 900 代才趋于稳定，说明过度改良初代个体可能导致种群过于集中，探索性下降，陷入局部最优。因此，在提升个体质量的同时，更需注重种群多样性的保持。

3. 从启发式优化角度思考 TSP 解法的可扩展性

TSP 是典型的 NP 难问题，依赖精确算法难以在合理时间内获得最优解。遗传算法作为一种典型的启发式优化方法，在本题中展现了较强的可扩展性和适应性。同时，通过动态调整参数（比如：种群个数，改良次数，进化次数，适者概率，生命强度，变异率）也能进一步提升算法的适应性与稳定性。

五、 参考资料

1. https://blog.csdn.net/CO_ocola/article/details/140065501
2. https://blog.csdn.net/m0_73473411/article/details/128967039